

Repository: <https://github.com/gmonzon05/Address-book-device>

Address Book Appliance (ABA) Implementation

Prepared by Ian Falkenstrom, Gregory Monzon, and Chris Najera

Table of Contents

1. Introduction
2. System Architecture
3. Build, Install, and Run Instructions
4. Functional Verification and Testing
5. Security Requirements and Controls
6. Issues Encountered
7. Lessons Learned

1. Introduction

This project implements the Address Book Appliance (ABA), a command-line application designed to manage address book records securely. The system is built to run on a Linux environment and follows a strict Command Line Interface (CLI) specification provided in the assignment.

The ABA supports multiple users with role-based access control, including an administrator account and normal users. The administrator is responsible for managing user accounts and viewing audit logs, while normal users can manage their own address book records.

The system emphasizes functional correctness, secure data handling, and strict adherence to the CLI specification. This includes matching exact command syntax, prompts, output messages, and completion codes required by the specification.

The system has been tested to ensure functional correctness, including proper execution of CLI commands, enforcement of user roles, correct handling of authentication, and accurate storage and retrieval of address book records.

2. System Architecture

The ABA system follows a modular architecture developed in the previous assignment. Each module is responsible for a specific part of the system, ensuring separation of concerns and improved maintainability.

The main modules are:

- CLI Parser: Handles user input, command parsing, and dispatching commands to the appropriate modules.
- Authentication Module: Manages login, logout, session tracking, password validation, and first-time password setup.
- User Manager: Handles user account creation, deletion, listing, and password resets.
- Record Manager: Manages address book records, including adding, editing, deleting, reading, importing, and exporting records.
- Access Control Module: Enforces role-based permissions, ensuring users can only perform allowed actions.
- Audit Logger: Records all security-relevant events such as logins, password changes, and record operations.
- Storage Module: Handles reading and writing the database to disk using encrypted storage.
- Crypto Module: Provides password hashing and encryption using secure algorithms.

All command behaviors, prompts, and output messages were implemented to strictly match the provided CLI specification.

3. Build, Install, and Run Instructions

Source Code Submission:

The complete and fully functional source code for the Address Book Application (ABA) is included directly in the submitted ZIP file. The GitHub repository referenced earlier in development may not reflect the final version used for testing and submission.

All functionality described in this document corresponds to the code included in the submission package.

Navigate to the project directory:

```
cd Address-book-device
```

Install required dependencies:

```
pip3 install -r requirements.txt
```

Run the application:

```
python3 aba.py
```

On startup, the program will display:

Address Book Application, version 1.0. Type "HLP" for commands.

The system uses the command prompt:

```
ABA>
```

The ABA system was also executed in the required Linux virtual machine environment to verify compatibility with the assignment platform.

4. Functional Verification and Testing

The Address Book Application (ABA) was tested to ensure that all required functionality operates correctly and matches the command-line interface specification. Testing was performed using Python 3 in a local development environment.

Authentication and User Management

The following sequence verifies login, password setup, and user creation.

The system was tested using the following commands:

```
ABA> LIN admin
```

```
Password:
```

```
You must set a new password before continuing.
```

```
New password:
```

```
Confirm new password:
```

```
OK
```

```
ABA> ADU user1
```

```
Re-enter your password to confirm:
```

```
OK
```

```
ABA> LOU
```

```
OK
```

```
ABA> LIN user1
```

```
Password:
```

```
You must set a new password before continuing.
```

```
New password:
```

```
Confirm new password:
```

```
OK
```

This confirms that authentication, session management, and password setup operate correctly.

Record Creation (ADR)

The following command was used to create a new record with all required fields:

```
ABA> ADR r1 SN=Smith GN=John PEM=john@example.com  
WEM=john@work.com PPH=123 WPH=456 SA=123Street  
CITY=SanDiego STP=CA CTY=USA PC=92101  
OK
```

This confirms that record creation, field validation, and data storage function correctly.

Record Retrieval (RER)

The stored record was successfully retrieved:

```
ABA> RER
```

RecordID: r1

sn: Smith

gn: John

pem: john@example.com

wem: john@work.com

pph: 123

wph: 456

sa: 123Street

city: SanDiego

stp: CA

cty: USA

pc: 92101

OK

This confirms that records are stored correctly and retrieved accurately.

Access Control Verification

The system correctly enforces role-based restrictions:

```
ABA> LIN admin
```

```
ABA> ADR r1 ...
```

```
Admin not authorized
```

This confirms that administrative users cannot manage records and that access control is functioning properly.

Error Handling Verification

The system correctly handles invalid input:

- Invalid login attempts return appropriate error messages
- Missing or invalid fields are rejected
- Unknown commands return:

```
Unrecognized command
```

Summary

These test cases demonstrate that the ABA system satisfies the required functional behavior, including authentication, role enforcement, record management, and correct CLI output formatting.

5. Security Requirements and Controls

The ABA system implements several important security requirements to ensure safe and controlled operation.

Identification

Each user is identified by a unique userID, which is used for login and tracking user activity.

Authentication

Users must authenticate using the LIN command with a valid password. Only one active session is allowed at a time.

First-Time Password Setup

When a new account is created, the user is required to create a password during the first login.

Password Security

Passwords are stored as secure hashes using the crypto module and are never stored in plain text.

Authorization

Role-based access control is enforced:

- Admin users can manage user accounts and view audit logs
- Normal users can manage their own address book records
- Admin users are not allowed to modify address book records

Access Control Enforcement

All commands verify authentication status and permissions before execution.

Audit Logging

All important events are recorded, including:

- Successful and failed logins
- Password changes
- User creation and deletion
- Record operations

Data Protection

The database is encrypted before being written to disk, protecting sensitive data.

Input Validation

User inputs are validated to prevent invalid data and potential injection attacks.

6. Issues Encountered

Several issues were encountered during the development of the ABA system.

One major issue was environment compatibility. The provided Linux VM used an older version of Python (3.5), while the codebase was written using newer Python features. This caused syntax errors and dependency installation problems.

Another challenge was ensuring strict compliance with the CLI specification. Small differences in output formatting, command syntax, and completion messages required careful debugging and adjustments.

The record structure required modification to match the official ABA field requirements, which differed from the initial simplified implementation.

Dependency installation issues were encountered due to outdated package repositories in the Linux VM.

7. Lessons Learned

This project provided several important lessons in software development and security.

Following a specification exactly is critical. Even small differences in output formatting or command behavior can lead to incorrect functionality.

Separating system components into modules improves organization, readability, and maintainability.

Security must be integrated into every part of the system. Authentication, authorization, encryption, and logging all play essential roles in protecting the application.

Environment compatibility is an important consideration. Code that works in one environment may fail in another due to differences in software versions and dependencies.

Testing incrementally and focusing on core functionality first helps manage complexity and ensures steady progress.